TUTORIAL

🚀 Tutorial Guiado: Implementación de Edge AI en STM32H7 (ARM Cortex-M7) con FreeRTOS

Este repositorio contiene la guía práctica y los pasos detallados para configurar, compilar e implementar un modelo de Inteligencia Artificial local (*Edge AI*) en una tarjeta de desarrollo **STM32H7** utilizando la terminal de Ubuntu. Diseñado para demostraciones académicas desde cero.

📋 1. Especificaciones de Hardware y Parámetros Técnicos

Responde a la lista de materiales y componentes necesarios.

- **Tarjeta de Desarrollo**: STM32H7 (Nucleo o Discovery con chip STM32H7 series)
- **Microcontrolador**: ARM Cortex-M7 a 480 MHz
- **Memoria de Hardware**: 2 MB Flash y 1 MB SRAM
- **Depurador/Programador**: ST-LINK integrado (vía USB)
- **Kernel de Tiempo Real**: FreeRTOS con modo *Tickless Idle* habilitado
- **Motor de Inferencia**: TensorFlow Lite for Microcontrollers (TFLite Micro)
- **Aceleración de Operadores**: Biblioteca **CMSIS-NN** y transferencias por **DMA de cero copia** (*Zero-Copy DMA*)
- **Modelo de IA**: Red Neuronal Convolucional (CNN) cuantizada en formato enteros de 8 bits (INT8)

📊 Métricas de Rendimiento Esperadas

- **Latencia de Inferencia Promedio**: 13.8 ms
- **Jitter Máximo de Latencia**: ±1.1 ms
- **Carga de CPU durante Inferencia**: 85%
- **Consumo Energético por Inferencia**: 0.92 mJ

🛠 2. Configuración del Entorno de Desarrollo en Ubuntu

Instrucciones detalladas de configuración inicial.

Abre la terminal de Ubuntu e instala las herramientas de compilación cruzada y depuración ejecutando el siguiente bloque:

```
# 1. Actualizar repositorios e instalar paquetes base
sudo apt update && sudo apt install -y \
    build-essential \
    cmake \
    ninja-build \
    git \
    python3 \
    screen \
    minicom \
    openocd

# 2. Instalación del compilador cruzado GCC para ARM y depurador compatible
sudo apt install -y gcc-arm-none-eabi binutils-arm-none-eabi gdb-multiarch

# 3. Verificación de versiones
arm-none-eabi-gcc --version
cmake --version
```

📁 3. Creación de la Estructura del Proyecto

Crea el directorio principal del repositorio y genera de forma automática toda la estructura de directorios:

```
# Crear la carpeta raíz y entrar en ella
mkdir -p Taller-RTOS
cd Taller-RTOS

# Crear la estructura de subdirectorios obligatorios
mkdir -p config src drivers/CMSIS
```

Estructura lógica resultante:

```
Taller-RTOS/
├── CMakeLists.txt          # Script de compilación CMake con Toolchain cruzado
├── config/
│   └── FreeRTOSConfig.h    # Configuración del Kernel FreeRTOS
├── src/
│   ├── main.cpp            # Tarea de inferencia TFLite Micro
│   ├── model_data.cpp      # Matriz de datos del modelo cuantizado
│   └── model_data.h        # Encabezado del modelo
└── drivers/
    └── CMSIS/              # Bibliotecas CMSIS y CMSIS-NN
```

🎁 4. Generación y Exportación del Modelo Cuantizado (INT8)

Crea un archivo binario ficticio de prueba y conviértelo directamente a C++ ejecutando estos comandos:

```
# 1. Crear un archivo binario de prueba de 10 KB
dd if=/dev/urandom of=cnn_quantized_model.tflite bs=1024 count=10

# 2. Convertirlo a matriz ejecutable C++ en src/model_data.cpp
xxd -i cnn_quantized_model.tflite > src/model_data.cpp

# 3. Crear el archivo de encabezado src/model_data.h automáticamente
cat << 'EOF' > src/model_data.h
#ifndef MODEL_DATA_H_
#define MODEL_DATA_H_

extern const unsigned char cnn_quantized_model_tflite[];
extern const unsigned int cnn_quantized_model_tflite_len;

#endif // MODEL_DATA_H_
EOF
```

⚙ 5. Configuración de FreeRTOS (`config/FreeRTOSConfig.h`)

Crea y escribe el archivo de configuración para el determinismo temporal:

```
cat << 'EOF' > config/FreeRTOSConfig.h
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

#define configUSE_PREEMPTION                    1
#define configUSE_TICKLESS_IDLE                 1   /* Reduce el jitter y el consumo energético */
```

```
#define configSUPPORT_STATIC_ALLOCATION         1   /* Asignación estática determinista de memoria */
#define configSUPPORT_DYNAMIC_ALLOCATION        0   /* Desactiva el heap dinámico */
#define configCPU_CLOCK_HZ                      ( 480000000UL ) /* 480 MHz en STM32H7 */
#define configTICK_RATE_HZ                      ( ( TickType_t ) 1000 )

#endif /* FREERTOS_CONFIG_H */
EOF
```

🧠 6. Código Comentado de Inferencia (`src/main.cpp`)

Código comentado del ejercicio práctico.

Crea el archivo `src/main.cpp` ejecutando este bloque:

```
cat << 'EOF' > src/main.cpp
#include "FreeRTOS.h"
#include "task.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "model_data.h"

// Reserva estática del Tensor Arena en SRAM (128 KB)
constexpr int kTensorArenaSize = 128 * 1024;
alignas(16) static uint8_t tensor_arena[kTensorArenaSize];

StaticTask_t xTaskBuffer;
StackType_t xStack[2048];

void vEdgeAITask(void *pvParameters) {
    // 1. Cargar el modelo cuantizado desde la memoria Flash
    const tflite::Model* model = tflite::GetModel(cnn_quantized_model_tflite);

    // 2. Registrar operadores optimizados acelerados por CMSIS-NN
    static tflite::MicroMutableOpResolver<5> micro_resolver;
    micro_resolver.AddConv2D();
    micro_resolver.AddAveragePool2D();
    micro_resolver.AddFullyConnected();
    micro_resolver.AddRelu();
    micro_resolver.AddReshape();

    // 3. Inicializar el intérprete de TFLite Micro
    tflite::ErrorReporter* error_reporter = tflite::GetDefaultErrorReporter();
    static tflite::MicroInterpreter interpreter(
        model, micro_resolver, tensor_arena, kTensorArenaSize, error_reporter);

    // Asignación estática de tensores
    if (interpreter.AllocateTensors() != kTfLiteOk) {
        while (1); // Bloqueo por fallo de asignación
    }

    TfLiteTensor* output = interpreter.output(0);

    for (;;) {
        // Ejecución de la inferencia en tiempo real
        if (interpreter.Invoke() == kTfLiteOk) {
            uint8_t top_class = output->data.uint8[0];
        }
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

int main(void) {
```

```
    xTaskCreateStatic(vEdgeAITask, "EdgeAI_Task", 2048, NULL, tskIDLE_PRIORITY + 2, xStack, &xTaskBuffer);
    vTaskStartScheduler();
    while (1);
}
EOF
```

🛠 7. Arquitectura de Compilación Cruzada (`CMakeLists.txt`)

Crea el archivo `CMakeLists.txt` en la raíz del proyecto para forzar el uso de la toolchain de ARM (`arm-none-eabi-gcc`):

```
cat << 'EOF' > CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
project(stm32h7_edgeai C CXX ASM)

# Forzar el compilador cruzado para ARM Cortex-M
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_CXX_COMPILER arm-none-eabi-g++)
set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)

set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)

set(CPU_FLAGS "-mcpu=cortex-m7 -mthumb -mfpu=fpv5-d16 -mfloat-abi=hard")
set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} ${CPU_FLAGS} -Wall -fdata-sections -ffunction-sections")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${CPU_FLAGS} -Wall -fdata-sections -ffunction-sections")

include_directories(
    config
    src
    drivers/CMSIS/Include
    drivers/CMSIS/NN/Include
)

file(GLOB_RECURSE SOURCES
    "src/*.cpp"
    "src/*.c"
)

add_executable(${PROJECT_NAME}.elf ${SOURCES})

add_custom_command(TARGET ${PROJECT_NAME}.elf POST_BUILD
    COMMAND arm-none-eabi-objcopy -O binary ${PROJECT_NAME}.elf ${PROJECT_NAME}.bin
    COMMAND arm-none-eabi-objcopy -O ihex ${PROJECT_NAME}.elf ${PROJECT_NAME}.hex
)
EOF
```

🔨 8. Compilación y Flasheo

Ejecuta los siguientes comandos para compilar el proyecto con Ninja:

```
mkdir -p build && cd build
cmake -G Ninja -DCMAKE_BUILD_TYPE=Release ..
ninja
```

(Para flashear físicamente la tarjeta con OpenOCD y un depurador ST-LINK conectado):

```
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg -c "program stm32h7_edgeai.elf verify reset exit"
```

⚠ 9. Troubleshooting y Errores Comunes

Sección de errores comunes y resolución de problemas encontrados durante el taller.

1. **Error:** `xxd: cnn_quantized_model.tflite: No such file or directory`
   - **Causa:** El archivo binario del modelo no se generó previamente en la carpeta de trabajo.
   - **Solución:** Ejecutar el comando con `dd` provisto en el Paso 4 para crear el archivo binario simulado de forma local.
2. **Error al compilar con banderas de hardware (`-mthumb`, `-mfpu`, etc.) usando GCC por defecto**
   - **Causa:** CMake está intentando compilar con el compilador nativo de la PC (`/usr/bin/c++`) en lugar del compilador cruzado.
   - **Solución:** Asegurar que el `CMakeLists.txt` contenga explícitamente `set(CMAKE_SYSTEM_NAME Generic)` y defina los compiladores `arm-none-eabi-gcc` / `arm-none-eabi-g++`.
3. **Error:** `fatal error: FreeRTOS.h: No such file or directory`
   - **Causa:** Las carpetas de dependencias externas (FreeRTOS kernel y TensorFlow Lite Micro) aún no se encuentran añadidas dentro de los directorios del proyecto o de inclusión.
   - **Solución:** Integrar los submódulos o directorios fuente correspondientes de FreeRTOS y TFLite Micro dentro de la estructura de `drivers/` o `Middlewares/` del proyecto embebido.